

LABORATORY

**LAB 15**

# Motor Speed Measurement

RPM Measurement with a Two-Channel Encoder Using EduFramework

S32K144 · MaaZEDU

# Contents

|     |                                             |    |
|-----|---------------------------------------------|----|
| 1   | Introduction .....                          | 2  |
| 1.1 | Lab Overview .....                          | 2  |
| 1.2 | Objectives .....                            | 2  |
| 2   | Background .....                            | 2  |
| 2.1 | Encoder in Motion Measurement Systems ..... | 2  |
| 2.2 | Two-Channel Incremental A/B Encoder .....   | 2  |
| 2.3 | Quadrature x4 Decoding and CPR .....        | 3  |
| 2.4 | RPM Calculation .....                       | 3  |
| 3   | Hardware Setup .....                        | 3  |
| 3.1 | Required Hardware .....                     | 3  |
| 3.2 | Motor Control Connections .....             | 4  |
| 3.3 | Motor and Encoder Connections .....         | 4  |
| 3.4 | Connection Diagram .....                    | 5  |
| 4   | EduFramework API .....                      | 5  |
| 4.1 | Encoder_Init() .....                        | 5  |
| 4.2 | Encoder_Reset() .....                       | 6  |
| 4.3 | Encoder_Update() .....                      | 6  |
| 4.4 | Encoder_GetRpm() .....                      | 6  |
| 5   | Lab Exercise .....                          | 7  |
| 5.1 | Requirements .....                          | 7  |
| 5.2 | Program .....                               | 7  |
| 5.3 | Program Explanation .....                   | 8  |
| 5.4 | Verification .....                          | 9  |
| 6   | Extension .....                             | 9  |
| 6.1 | Adjusting the PWM Level through UART .....  | 9  |
| 7   | References .....                            | 10 |

# 1 Introduction

## 1.1 Lab Overview

Lab 14 used a motor driver module to configure the PWM level, direction, and stop state of a DC motor. However, the PWM value represents only the control command generated by the microcontroller and does not directly indicate the actual mechanical speed of the motor.

In systems that require motion observation, an encoder can be used to provide feedback from a rotating shaft. By counting encoder state transitions during a defined time interval, the program can calculate rotational speed in revolutions per minute (RPM). A two-channel encoder can also determine the rotation direction from the phase relationship between the two signals [3] [5] .

This lab adds a two-channel A/B encoder to the motor system introduced in Lab 14. The motor is driven continuously at a fixed PWM level, while EduFramework measures the encoder signals and sends the actual RPM value to the Serial Monitor through `Serial1` .

## 1.2 Objectives

### OBJECTIVES

The objectives of this laboratory are:

1. Describe the role of an encoder in measuring rotational motion of a motor.
2. Explain the basic operating principle of a two-channel incremental A/B encoder and quadrature x4 decoding.
3. Determine the number of counts per revolution from the encoder pulse specification.
4. Use the Encoder Device API to update and read the motor RPM.
5. Display the motor speed in real time on the Serial Monitor.

# 2 Background

## 2.1 Encoder in Motion Measurement Systems

An encoder is a device that generates electrical signals corresponding to mechanical motion. In rotating systems, an encoder is commonly attached to a motor shaft to provide information about position, motion count, direction, or speed [1] [5] .

Encoders can use different sensing principles. Common technologies include optical, magnetic, and capacitive encoders [6] . This lab uses an optical encoder integrated directly with the DC motor.

In an optical encoder, a disk containing periodic slots or patterns rotates with the motor shaft. A light source and optical receiver convert the variation in light into a sequence of electrical pulses. A two-channel arrangement allows the system to determine not only motion count but also the direction of rotation [2] .

## 2.2 Two-Channel Incremental A/B Encoder

The encoder used in this lab provides two channels, A and B. These signals are periodic and phase-shifted relative to each other. The order in which A and B change depends on the direction of rotation, allowing the decoder to determine motion in both directions [3] .

The channel states can be visualized as logic pairs:

00 → 01 → 11 → 10 → 00

When the rotation direction reverses, the transition order also reverses. EduFramework tracks valid state transitions and increments or decrements the encoder count accordingly [8].

### 2.3 Quadrature x4 Decoding and CPR

A complete cycle of a two-channel encoder contains four edge transitions that can be counted. When both rising and falling edges of channels A and B are used, the method is referred to as quadrature x4 decoding [3]. The encoder integrated with the motor used in this lab produces 30 pulses per revolution. EduFramework uses x4 decoding, so the number of counts per mechanical revolution is:

$$30 \times 4 = 120$$

Therefore, the `countsPerRevolution` value passed to `Encoder_Init()` is 120U.

#### Note

*The CPR value passed to the Encoder Device API must represent the number of decoded counts per mechanical revolution, not only the nominal number of encoder pulses.*

### 2.4 RPM Calculation

Rotational speed can be calculated from the number of counts that change during a defined time interval. The general principle is to measure the count change within a sampling window, then convert the result using CPR and the elapsed time to obtain revolutions per minute [4].

In EduFramework, the calculation can be represented by the following relationship:

$$\text{RPM} = |\Delta\text{Count}| \times 60000 / (\text{CPR} \times \Delta t_{\text{ms}})$$

where:

- `ΔCount` is the count change during the measurement window;
- `CPR` is the number of counts per revolution;
- `Δt_ms` is the measurement-window duration in milliseconds;
- `60000` is the number of milliseconds in one minute.

RPM is returned as a positive magnitude. Rotation direction is managed separately by the Encoder Device API.

## 3 Hardware Setup

### 3.1 Required Hardware

Lab 15 continues to use the motor-control system from Lab 14 and adds the encoder integrated with the motor. The TB6612FNG module and external power supply continue to provide the motor power-control stage.

| Hardware                      | Description                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------|
| MaaZEDU Development Board     | Development board based on the S32K144 microcontroller.                                  |
| TB6612FNG Motor Driver Module | Motor driver module previously used in Lab 14.                                           |
| DC Motor with Encoder         | DC motor with an integrated two-channel A/B optical encoder, 30 pulses per revolution.   |
| External Motor Power Supply   | External supply that powers the motor through the TB6612FNG.                             |
| Jumper wires                  | Connect the control, encoder, and power signals.                                         |
| USB Cable                     | Connects MaaZEDU to the computer for program uploading and Serial Monitor communication. |

Table 15.3.1. Hardware used in this lab

### 3.2 Motor Control Connections

The motor-control connections remain unchanged from Lab 14:

| Component      | Logical Pin | MCU Pin | Function       |
|----------------|-------------|---------|----------------|
| TB6612FNG PWMA | GPI02       | PTD14   | PWM Output     |
| TB6612FNG AIN1 | GPI03       | PTD13   | Digital Output |
| TB6612FNG AIN2 | GPI04       | PTD12   | Digital Output |
| TB6612FNG STBY | GPI05       | PTD11   | Digital Output |

Table 15.3.2. Motor-control signals reused from Lab 14

The H-bridge, VCC supply, VM supply, and PWM-control details were covered in Lab 14 and are therefore not repeated in this lab.

### 3.3 Motor and Encoder Connections

The motor used in this lab has a six-pin connector. According to the verified hardware configuration, the pins are connected as follows:

| Motor Pin | Connection | Function               |
|-----------|------------|------------------------|
| Pin 1     | A01        | First motor terminal.  |
| Pin 2     | A02        | Second motor terminal. |
| Pin 3     | 3.3V       | Encoder power supply.  |
| Pin 4     | GND        | Encoder ground.        |
| Pin 5     | GPI08      | Encoder Channel A.     |
| Pin 6     | GPI09      | Encoder Channel B.     |

GPI08 and GPI09 are selected as the two encoder inputs because these Logical Pins support Digital Input and external interrupt capability in the current EduFramework configuration [7] [8] .

### 3.4 Connection Diagram

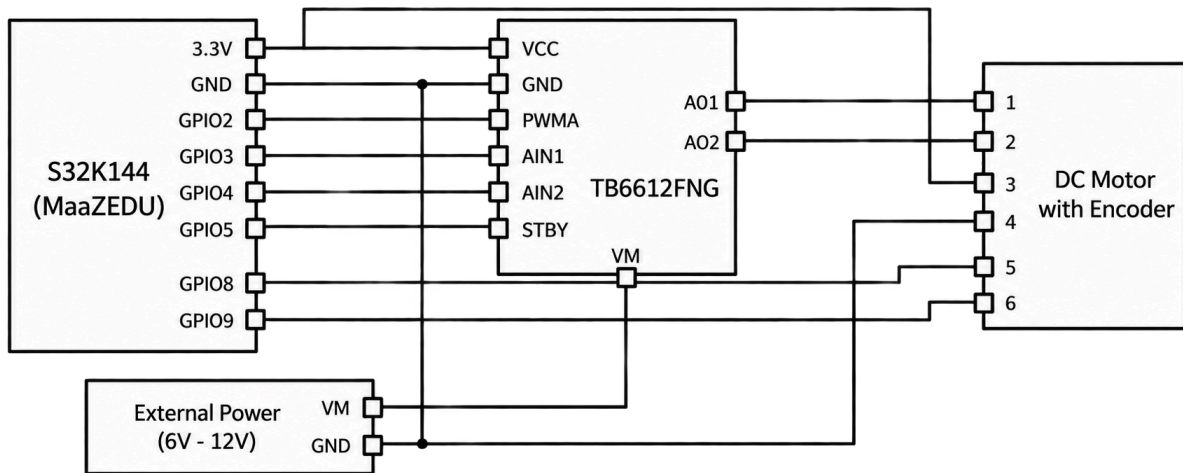


Figure 15.3.1. Connection diagram for the DC motor with integrated encoder and MaaZEDU

## 4 EduFramework API

Lab 15 reuses the DC Motor Device API from Lab 14 to create motion and focuses on the new Encoder Device API. The main APIs used in the exercise are `Encoder_Init()`, `Encoder_Reset()`, `Encoder_Update()`, and `Encoder_GetRpm()` [8].

### 4.1 `Encoder_Init()`

`Encoder_Init()` initializes the two encoder channels and configures the number of counts per revolution.

#### `Encoder_Init`

Initialize a two-channel encoder and configure the number of decoded counts per mechanical revolution.

#### Syntax

```
Encoder_Init(channelAPin, channelBPin, countsPerRevolution);
```

#### Parameters

| Parameter                        | Type / accepted value | Description                                         |
|----------------------------------|-----------------------|-----------------------------------------------------|
| <code>channelAPin</code>         | Logical Pin           | Pin connected to encoder Channel A.                 |
| <code>channelBPin</code>         | Logical Pin           | Pin connected to encoder Channel B.                 |
| <code>countsPerRevolution</code> | Count / revolution    | Number of decoded counts per mechanical revolution. |

#### Return value

Returns `1U` when initialization is successful and `0U` when the configuration is invalid.

Example:

```
Encoder_Init(GPI08, GPI09, 120U);
```

## 4.2 Encoder\_Reset()

`Encoder_Reset()` resets the accumulated count, RPM value, and rotation direction state to their initial values.

### Encoder\_Reset

Reset the encoder measurement state.

#### Syntax

```
Encoder_Reset();
```

#### Return value

No return value.

## 4.3 Encoder\_Update()

`Encoder_Update()` updates the speed measurement using the encoder count and elapsed time. The function is non-blocking and should be called repeatedly from the main loop.

### Encoder\_Update

Update the latest encoder RPM and rotation direction.

#### Syntax

```
Encoder_Update();
```

#### Return value

No return value.

In the current EduFramework version, RPM is updated using an internal measurement window. The application can continue performing other tasks between updates.

## 4.4 Encoder\_GetRpm()

`Encoder_GetRpm()` returns the latest rotational speed calculated by `Encoder_Update()`.

### Encoder\_GetRpm

Read the latest encoder rotational speed.

#### Syntax

```
Encoder_GetRpm();
```

#### Return value

Rotational speed in revolutions per minute (RPM).

EduFramework also provides `Encoder_GetCount()` and `Encoder_GetDirection()` to read the accumulated count and rotation direction. These APIs are not required in the main exercise but can be used when additional encoder state information is needed.

## 5 Lab Exercise

---

### 5.1 Requirements

Build a program that runs the motor continuously at approximately 50% control level, measures the speed using the encoder, and displays the RPM on the Serial Monitor.

The program must meet the following requirements:

- Initialize Serial1 at 9600 bps .
- Initialize the motor-control system using GPIO2 through GPIO5 .
- Initialize the encoder with Channel A on GPIO8 , Channel B on GPIO9 , and 120 counts per revolution.
- Set the motor speed-control value to 128 .
- Run the motor continuously in the Forward direction.
- Call Encoder\_Update() continuously in the main loop.
- Display the RPM on the Serial Monitor every 200 ms .

### 5.2 Program

In src/main.c , implement the program as follows:

```
#include "Arduino.h"
#include "hardware_serial.h"
#include "dc_motor.h"
#include "encoder.h"

int main(void)
{
    uint32_t previousTime = 0U;

    setup();

    Serial1_begin(9600U);

    DCMotor_Init(GPIO2, GPIO3, GPIO4, GPIO5);

    if (0U == Encoder_Init(GPIO8, GPIO9, 120U))
    {
        Serial1_println("Encoder initialization failed.");

        while (1)
        {
        }
    }

    Encoder_Reset();

    DCMotor_SetSpeed(128U);
    DCMotor_Forward();

    Serial1_println("Motor speed measurement started.");

    while (1)
    {
        Encoder_Update();

        if ((millis() - previousTime) >= 200U)
        {
            previousTime = millis();

            Serial1_print("RPM: ");
            Serial1_printFloat(Encoder_GetRpm());
            Serial1_println("");
        }
    }

    return 0;
}
```

*Listing 15.5.1. Motor speed measurement program using an encoder*

### 5.3 Program Explanation

After `setup()`, `Serial1_begin(9600U)` initializes the Serial Monitor. The motor control section continues to use the configuration introduced in Lab 14.

`Encoder_Init(GPI08, GPI09, 120U)` configures the two encoder channels and declares `120` counts per revolution. If initialization fails, the program prints an error message and does not continue to the measurement stage.

After `Encoder_Reset()` resets the measurement state, `DCMotor_SetSpeed(128U)` sets the control level to approximately `50%`, and `DCMotor_Forward()` keeps the motor rotating continuously.

Inside the main loop, `Encoder_Update()` is called continuously to update the speed measurement. Every `200 ms`, the program reads the latest value using `Encoder_GetRpm()` and sends the result to the Serial Monitor.

Separating the Serial update timing from the encoder measurement allows the main loop to continue calling `Encoder_Update()` frequently instead of blocking the program with `delay()`.

## 5.4 Verification

Build and upload the program to the MaaZEDU Development Board. Apply external power to the motor, open the Serial Monitor at `9600 bps`, and observe the output.

Example:

```
Motor speed measurement started.  
RPM: 2115.000  
RPM: 2130.000  
RPM: 2125.000  
RPM: 2130.000
```

The actual RPM depends on the motor, supply voltage, mechanical load, and operating conditions, so it is not required to match the example exactly.

### EXPECTED RESULT

The motor rotates continuously in one direction at control value `128`. The Serial Monitor displays the RPM measured by the encoder and updates the value periodically at approximately `200 ms`. When the motor speed becomes stable, consecutive RPM values fluctuate around a relatively stable level.

## 6 Extension

The main exercise uses a fixed PWM level to focus on encoder-based speed measurement. By combining UART input data with RPM feedback, the program can be extended into an interactive motor-speed control application.

### 6.1 Adjusting the PWM Level through UART

Extend the program so that the user can enter a value from `0` to `255` through the Serial Monitor. The received value is used as the new control level for `DCMotor_SetSpeed()`.

After each change in the control value, the encoder continues to measure the actual speed and the Serial Monitor displays the corresponding RPM.

The intended processing flow is:

UART Input → PWM Command → DC Motor → Encoder → Measured RPM

Hints:

- Check incoming data from `Serial1`.
- Convert the received data into a numeric value.
- Verify that the value is within the `0..255` range.
- Update the control level using `DCMotor_SetSpeed()`.
- Continue calling `Encoder_Update()` in the main loop.

- Display both the current PWM value and the actual RPM for comparison.

An automatic algorithm that adjusts PWM according to RPM is not required. The user directly changes the control command and observes the measured motor-speed response.

---

#### EXPECTED RESULT

The user can enter different control levels through the Serial Monitor. The motor speed changes accordingly, and the encoder provides the actual RPM so that the effect of each PWM level can be observed.

## 7 References

---

- [1] University of Michigan, *EECS 461 - Embedded Control Systems*, Department of Electrical Engineering and Computer Science.
- [2] Eindhoven University of Technology, *Incremental Optical Encoder and Position Measurement*, Technical Publication.
- [3] National Instruments, *Hardware Encoder Measurements: How-To Guide*, NI Technical Documentation.
- [4] Texas Instruments, *eQEP Position and Speed Measurement*, MCU+ SDK Documentation.
- [5] Rockwell Automation, *Encoder/Counter Modules User Manual*, Industrial Automation Documentation.
- [6] CUI Devices, *Capacitive, Magnetic, and Optical Encoders: Comparing the Technologies*, Technical Article.
- [7] MaaZEDU, *MaaZEDU Development Board Guide*, Rev. 1.0, 2025.
- [8] EduFramework, *Encoder Device API*, EduFramework Source Code.